claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_ace7351e-c4e\agent-a610073d60f054c42.jsonl`
- SHA-256: `77fb3b8f11f17fd85f8eda5cac6903d9569c9572bb907a60dbc558c80cdb63eb`
- Source modified: `2026-07-06T22:18:56+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf_ace7351e-c4e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T22:16:32.966Z)

```
Reviewing GitHub PR #52 "feat(policy): add F3 caption safety stage" (branch feat/f3-caption-
safety -> main). ADR 0012 F3.
+299/-41, 4 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr52 (read there or 'git -C
C:/Users/avidu/Projects/Telegram show origin/feat/f3-caption-safety:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F3 DOES:
- policy.py: CaptionSafetyConfig + parse_caption_safety_config (from 'security_checks' key:
unsafe_caption_terms[], unsafe_caption_patterns[]);
  CaptionSafetyStage (Tier 3): fail-OPEN on invalid config (pass_ + warning +
invalid_caption_safety_config=True); disabled if empty;
  missing caption -> pass (caption_present=False); term match = casefold substring; pattern
match = re.search(pattern, caption) (case-SENSITIVE
  unless user adds (?i)); regexes validated via re.compile at parse time (invalid regex ->
ValueError -> fail-open).
- telethon_client.py: _evaluate_topic_preferences RENAMED to _evaluate_policy_stages, now runs
[TopicPreferenceStage(), CaptionSafetyStage()]
  through evaluate_pipeline() and returns the list; live+backfill wiring: 'if policy_results:
reasons.extend(...); matched = policy_results[-1].passed'.
- Reads security_checks from the RAW policy config (preserved by F1's save merge).
on_new_message/backfill_user are '# pragma: no cover'.

CONFIRMED by the main reviewer: gate green (523 passed @ 95.77%); ReDoS is REAL ? the valid
pattern '(a+)+$' on a 40-char caption hangs
re.search past a 6s timeout (rc=124), blocking the synchronous event loop. Feature is dormant
today (no command sets security_checks).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario, ranked.
Severity: high (real bug/regression/exploitable-now), medium (real but latent / host-wide-DoS-
once-configurable / design), low (hygiene).

DIMENSION: evaluate_pipeline migration correctness + config parsing.
Investigate:
- The wiring changed from 'matched = topic_result.passed' (F2) to 'matched =
policy_results[-1].passed' over evaluate_pipeline([topic, caption]).
  Verify this is EQUIVALENT-or-better: does policy_results[-1].passed correctly equal the AND of
both stages GIVEN evaluate_pipeline short-circuits?
  (results[-1] is the first failure OR the final pass). Is it fragile ? would it break if
evaluate_pipeline stopped short-circuiting? evaluate_pipeline
  returns (passed, results) but _evaluate_policy_stages DISCARDS passed and callers use
results[-1].passed ? should it use the aggregate 'passed'?
```

```
  Confirm topic-stage F2 behavior is unchanged by the migration (short-circuit: caption doesn't
run if topic fails).
- Edge: policy_results empty (policy None) -> callers guard 'if policy_results'. Any case where
results is non-empty but [-1] is the wrong stage?
- parse_caption_safety_config / _string_tuple edges: security_checks non-dict, terms/patterns
non-list, non-string items, empty-after-strip, duplicate,
  a term that is empty string. Missing caption vs empty-after-strip caption. casefold term
matching false positives (substring).
- Does load_caption_safety_config read the same raw config as topic (consistency)? Does it
round-trip with F1 save (security_checks preserved)?
Run repros. Report findings.
```

2. assistant (2026-07-06T22:16:35.568Z)

```
I'll investigate the evaluate_pipeline migration correctness and config parsing dimensions. Let
me start by reading the relevant files.
```

3. user (2026-07-06T22:16:37.053Z)

```
tg_video_filter
```

4. assistant (2026-07-06T22:16:38.993Z)

```
I'll investigate the migration and config parsing. Let me read the key files.
```

5. user (2026-07-06T22:16:41.820Z)

```
8e7acb6 feat(policy): add caption safety stage
6d3d7a6 feat(policy): add F2 topic preferences (#51)
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
---DIFF---
 src/tg_video_filter/backfill.py        |  10 +--
 src/tg_video_filter/policy.py          | 139 ++++++++++++++++++++++++---
 src/tg_video_filter/telethon_client.py |  32 ++++---
 tests/test_policy.py                   | 159 +++++++++++++++++++++++++++++++++
 4 files changed, 299 insertions(+), 41 deletions(-)
```

6. user (2026-07-06T22:16:45.229Z)

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index 86db324..9e2664c 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -9,6 +9,7 @@ from __future__ import annotations

 import json
 import logging
+import re
 from dataclasses import dataclass, field
 from typing import Protocol

@@ -77,6 +78,18 @@ class TopicPreferences:
         return bool(self.include or self.exclude)


+@dataclass(frozen=True)
+class CaptionSafetyConfig:
+    """Parsed ADR 0012 caption safety config."""
+
+    unsafe_terms: tuple[str, ...] = ()
+    unsafe_patterns: tuple[str, ...] = ()
+
+    @property
+    def enabled(self) -> bool:
```

```
+        return bool(self.unsafe_terms or self.unsafe_patterns)
+
+
 def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
     """Parse ``topics`` from a raw policy config object."""
     raw_topics = config.get("topics")
@@ -84,8 +97,8 @@ def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
         return TopicPreferences()
     if not isinstance(raw_topics, dict):
         raise ValueError("topics must be a JSON object")
-    include = _string_tuple(raw_topics.get("include"))
-    exclude = _string_tuple(raw_topics.get("exclude"))
+    include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
+    exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
     if not include and not exclude:
         return TopicPreferences()
     mode = raw_topics.get("mode", "any")
@@ -98,6 +111,40 @@ def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
     )


+def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
+    """Parse caption safety settings from ``security_checks``.
+
+    Expected shape:
+    ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
+    """
+    raw_security = config.get("security_checks")
+    if raw_security is None:
+        return CaptionSafetyConfig()
+    if not isinstance(raw_security, dict):
+        raise ValueError("security_checks must be a JSON object")
+
+    unsafe_terms = _string_tuple(
+        raw_security.get("unsafe_caption_terms"),
+        field_name="security_checks.unsafe_caption_terms",
+    )
+    unsafe_patterns = _string_tuple(
+        raw_security.get("unsafe_caption_patterns"),
+        field_name="security_checks.unsafe_caption_patterns",
+    )
+    for pattern in unsafe_patterns:
+        try:
+            re.compile(pattern)
+        except re.error as exc:
+            raise ValueError(
+                "security_checks.unsafe_caption_patterns contains invalid regex "
+                f"{pattern!r}: {exc}"
+            ) from exc
+    return CaptionSafetyConfig(
+        unsafe_terms=unsafe_terms,
+        unsafe_patterns=unsafe_patterns,
+    )
+
+
 def load_topic_preferences(policy: Policy) -> TopicPreferences:
     """Load topic preferences from ``policy.config_json``."""
     decoded = json.loads(policy.config_json)
@@ -106,19 +153,27 @@ def load_topic_preferences(policy: Policy) -> TopicPreferences:
     return parse_topic_preferences(decoded)


-def _string_tuple(value: object) -> tuple[str, ...]:
+def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
+    """Load caption safety settings from ``policy.config_json``."""
```

```
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return parse_caption_safety_config(decoded)
+
+
+def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
     if value is None:
         return ()
     if not isinstance(value, list):
-        raise ValueError("topic include/exclude values must be arrays")
-    topics: list[str] = []
+        raise ValueError(f"{field_name} must be an array")
+    values: list[str] = []
     for item in value:
         if not isinstance(item, str):
-            raise ValueError("topic include/exclude arrays must contain strings")
-        topic = item.strip()
-        if topic:
-            topics.append(topic)
-    return tuple(topics)
+            raise ValueError(f"{field_name} must contain strings")
+        text = item.strip()
+        if text:
+            values.append(text)
+    return tuple(values)


 class TopicPreferenceStage:
@@ -189,6 +244,66 @@ def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str,
...]:
     return tuple(topic for topic in topics if topic.casefold() in haystack)


+class CaptionSafetyStage:
+    """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
+
+    stage_name = "caption-safety"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            config = load_caption_safety_config(ctx.policy)
+        except (json.JSONDecodeError, ValueError) as exc:
+            log.warning(
+                "caption safety disabled due to invalid config policy_id=%s",
+                ctx.policy.id,
+                exc_info=True,
+            )
+            return StageResult.pass_(
+                self.stage_name,
+                f"invalid caption safety config ignored: {exc}",
+                invalid_caption_safety_config=True,
+            )
+        if not config.enabled:
+            return StageResult.pass_(self.stage_name, "caption safety disabled")
+
+        caption = (ctx.content_item.text or "").strip()
+        if not caption:
+            return StageResult.pass_(
+                self.stage_name,
+                "no caption to scan",
+                caption_present=False,
+            )
+
+        haystack = caption.casefold()
```

```
+        for term in config.unsafe_terms:
+            if term.casefold() in haystack:
+                return StageResult.fail(
+                    self.stage_name,
+                    f"unsafe caption term matched: {term}",
+                    caption_present=True,
+                    match_type="term",
+                    matched_caption_term=term,
+                )
+
+        for pattern in config.unsafe_patterns:
+            if re.search(pattern, caption):
+                return StageResult.fail(
+                    self.stage_name,
+                    "unsafe caption pattern matched",
+                    caption_present=True,
+                    match_type="pattern",
+                    matched_caption_pattern=pattern,
+                )
+
+        return StageResult.pass_(
+            self.stage_name,
+            "caption passed safety checks",
+            caption_present=True,
+            checked_terms=len(config.unsafe_terms),
+            checked_patterns=len(config.unsafe_patterns),
+        )
+
+
 async def evaluate_pipeline(
     ctx: PolicyContext,
     stages: list[PolicyStage],
@@ -204,12 +319,16 @@ async def evaluate_pipeline(


 __all__ = [
+    "CaptionSafetyConfig",
+    "CaptionSafetyStage",
     "PolicyContext",
     "PolicyStage",
     "StageResult",
     "TopicPreferenceStage",
     "TopicPreferences",
     "evaluate_pipeline",
+    "load_caption_safety_config",
     "load_topic_preferences",
+    "parse_caption_safety_config",
     "parse_topic_preferences",
 ]
```

7. assistant (2026-07-06T22:16:47.580Z)

Now let me read the full evaluate_pipeline and the telethon_client/backfill wiring.

8. user (2026-07-06T22:16:48.369Z)

```
300                 "caption passed safety checks",
301                 caption_present=True,
302                 checked_terms=len(config.unsafe_terms),
303                 checked_patterns=len(config.unsafe_patterns),
304             )
305
306
307     async def evaluate_pipeline(
308         ctx: PolicyContext,
```

```
309         stages: list[PolicyStage],
310     ) -> tuple[bool, list[StageResult]]:
311         """Evaluate stages in order and short-circuit on the first failure."""
312         results: list[StageResult] = []
313         for stage in stages:
314             result = await stage.evaluate(ctx)
315             results.append(result)
316             if not result.passed:
317                 return False, results
318         return True, results
319
320
321     __all__ = [
322         "CaptionSafetyConfig",
323         "CaptionSafetyStage",
324         "PolicyContext",
325         "PolicyStage",
326         "StageResult",
327         "TopicPreferenceStage",
328         "TopicPreferences",
329         "evaluate_pipeline",
330         "load_caption_safety_config",
331         "load_topic_preferences",
332         "parse_caption_safety_config",
333         "parse_topic_preferences",
334     ]
335
```

9. user (2026-07-06T22:16:52.290Z)

```
diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
index 15a7aa2..64905bf 100644
--- a/src/tg_video_filter/backfill.py
+++ b/src/tg_video_filter/backfill.py
@@ -34,7 +34,7 @@ from tg_video_filter.filters import FilterEngine
 from tg_video_filter.models import Channel, DeliveryMode, User, VideoRecord
 from tg_video_filter.telethon_client import (
     _content_item_preview,
-    _evaluate_topic_preferences,
+    _evaluate_policy_stages,
     _extract_document_id,
     _message_text,
     _peer_id,
@@ -244,7 +244,7 @@ async def backfill_user(  # pragma: no cover - requires live Telethon
connection
                 matched = result.matched
                 reasons = list(result.reasons)
                 if matched and source is not None:
-                    topic_result = await _evaluate_topic_preferences(
+                    policy_results = await _evaluate_policy_stages(
                         conn,
                         user.id,
                         source,
@@ -258,9 +258,9 @@ async def backfill_user(  # pragma: no cover - requires live Telethon
connection
                             matched,
                         ),
                     )
-                    if topic_result is not None:
-                        reasons.append(topic_result.reason)
-                        matched = topic_result.passed
+                    if policy_results:
+                        reasons.extend(result.reason for result in policy_results)
+                        matched = policy_results[-1].passed
```

```
                record = VideoRecord(
                    user_id=user.id,
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 7b393f8..6dad7c4 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -44,7 +44,13 @@ from tg_video_filter.models import (
     User,
     VideoRecord,
 )
-from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
+from tg_video_filter.policy import (
+    CaptionSafetyStage,
+    PolicyContext,
+    StageResult,
+    TopicPreferenceStage,
+    evaluate_pipeline,
+)

 log = logging.getLogger(__name__)

@@ -272,7 +278,7 @@ def make_new_message_handler(
         matched = result.matched
         reasons = list(result.reasons)
         if matched and source is not None:
-            topic_result = await _evaluate_topic_preferences(
+            policy_results = await _evaluate_policy_stages(
                 conn,
                 user.id,
                 source,
@@ -286,9 +292,9 @@ def make_new_message_handler(
                     matched,
                 ),
             )
-            if topic_result is not None:
-                reasons.append(topic_result.reason)
-                matched = topic_result.passed
+            if policy_results:
+                reasons.extend(result.reason for result in policy_results)
+                matched = policy_results[-1].passed

         record = VideoRecord(
             user_id=user.id,
@@ -420,19 +426,21 @@ def _content_item_preview(
     )


-async def _evaluate_topic_preferences(
+async def _evaluate_policy_stages(
     conn: aiosqlite.Connection,
     account_id: int,
     source: Source,
     content_item: ContentItem,
-) -> StageResult | None:
-    """Evaluate ADR 0012 topic preferences for a candidate item."""
+) -> list[StageResult]:
+    """Evaluate ADR 0012 policy stages for a candidate item."""
     policy = await db.get_default_policy(conn, account_id)
     if policy is None:
-        return None
-    return await TopicPreferenceStage().evaluate(
-        PolicyContext(content_item=content_item, source=source, policy=policy)
+        return []
+    _passed, results = await evaluate_pipeline(
+        PolicyContext(content_item=content_item, source=source, policy=policy),
```

```
+        [TopicPreferenceStage(), CaptionSafetyStage()],
     )
+    return results


 def _peer_id(peer: object) -> int:
@@ -468,6 +476,6 @@ __all__ = [
     "extract_video_meta",
     "make_new_message_handler",
     "_content_item_preview",
-    "_evaluate_topic_preferences",
+    "_evaluate_policy_stages",
     "_message_text",
 ]
```

10. assistant (2026-07-06T22:16:58.151Z)

Now I have the full picture. Let me analyze the key correctness question: does `policy_results[-1].passed` correctly equal the AND of both stages given short-circuit semantics.

Let me verify the short-circuit logic. `evaluate_pipeline` appends each result and returns on first failure. So:
- Topic fails ? results = [topic_fail], `[-1].passed` = False. Correct (AND = False). Caption doesn't run. ?
- Topic passes, caption fails ? results = [topic_pass, caption_fail], `[-1].passed` = False. Correct. ?
- Both pass ? results = [topic_pass, caption_pass], `[-1].passed` = True. Correct. ?

So `results[-1].passed` equals the aggregate `passed` in all cases (the last appended result is either the first failure or the final pass). It's semantically equivalent to `_passed`. It's fragile (discarding the actual `passed` return value and reconstructing from `[-1]`), but not currently a bug.

Let me check the F2 behavior for topic stage and confirm nothing changed, and look at StageResult and the reasons handling.

11. user (2026-07-06T22:16:59.381Z)

```
1       """Policy pipeline primitives for ADR 0012.
2
3       The policy layer is deliberately small and framework-free in F1: stages receive
4       an immutable context, return a structured result, and the evaluator stops on
5       the first failure. Later Track F phases add concrete stages.
6       """
7
8       from __future__ import annotations
9
10      import json
11      import logging
12      import re
13      from dataclasses import dataclass, field
14      from typing import Protocol
15
16      from tg_video_filter.models import ContentItem, Policy, Source
17
18      log = logging.getLogger(__name__)
19
20
21      @dataclass(frozen=True)
22      class PolicyContext:
23          """Immutable snapshot available to a policy stage."""
24
25          content_item: ContentItem
26          source: Source
```

```
27          policy: Policy
28          fetcher: object | None = None
29          budget: object | None = None
30
31
32      @dataclass(frozen=True)
33      class StageResult:
34          """Outcome of one policy stage."""
35
36          passed: bool
37          reason: str
38          stage_name: str
39          metadata: dict[str, object] = field(default_factory=dict)
40
41          @staticmethod
42          def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
43              return StageResult(
44                  passed=True,
45                  reason=reason,
46                  stage_name=stage_name,
47                  metadata=metadata,
48              )
49
50          @staticmethod
51          def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
52              return StageResult(
53                  passed=False,
54                  reason=reason,
55                  stage_name=stage_name,
56                  metadata=metadata,
57              )
58
59
60      class PolicyStage(Protocol):
61          """A single stateless gate in the policy pipeline."""
62
63          async def evaluate(self, ctx: PolicyContext) -> StageResult:
64              """Evaluate ``ctx`` and return a stage result."""
65              ...
66
67
68      @dataclass(frozen=True)
69      class TopicPreferences:
70          """Parsed ADR 0012 topic preference config."""
71
72          include: tuple[str, ...] = ()
73          exclude: tuple[str, ...] = ()
74          mode: str = "any"
75
76          @property
77          def enabled(self) -> bool:
78              return bool(self.include or self.exclude)
79
80
```

12. assistant (2026-07-06T22:17:03.908Z)

Now let me look at the reasons handling difference. In F2, only the topic reason was appended. Now, `reasons.extend(result.reason for result in policy_results)`. Let me examine what reasons look like when both stages pass ? this changes the reasons list content. Let me check the TopicPreferenceStage to understand its pass reason.

13. user (2026-07-06T22:17:04.883Z)

```
198             if not prefs.enabled:
```

```
199                 return StageResult.pass_(self.stage_name, "topic preferences disabled")
200
201             haystack = _topic_haystack(ctx)
202             matched_excludes = _matched_topics(prefs.exclude, haystack)
203             if matched_excludes:
204                 return StageResult.fail(
205                     self.stage_name,
206                     f"excluded topic matched: {matched_excludes[0]}",
207                     matched_exclude_topics=list(matched_excludes),
208                 )
209
210             if not prefs.include:
211                 return StageResult.pass_(self.stage_name, "no include topics configured")
212
213             matched_includes = _matched_topics(prefs.include, haystack)
214             if prefs.mode == "any":
215                 if matched_includes:
216                     return StageResult.pass_(
217                         self.stage_name,
218                         f"matched include topic: {matched_includes[0]}",
219                         matched_include_topics=list(matched_includes),
220                     )
221                 return StageResult.fail(self.stage_name, "no include topics matched")
222
223             missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
224             if missing:
225                 return StageResult.fail(
226                     self.stage_name,
227                     f"missing required topic: {missing[0]}",
228                     matched_include_topics=list(matched_includes),
229                     missing_include_topics=list(missing),
230                 )
231             return StageResult.pass_(
232                 self.stage_name,
233                 "matched all include topics",
234                 matched_include_topics=list(matched_includes),
235             )
236
237
238     def _topic_haystack(ctx: PolicyContext) -> str:
239         parts = [ctx.source.title or "", ctx.content_item.text or ""]
240         return "\n".join(parts).casefold()
241
242
243     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
244         return tuple(topic for topic in topics if topic.casefold() in haystack)
245
246
247     class CaptionSafetyStage:
```

14. assistant (2026-07-06T22:17:12.308Z)

Now let me check a subtle correctness issue: the `matched = policy_results[-1].passed` reconstruction. There's a real concern here. When the topic stage passes but caption safety is *disabled* (config not set - the dormant-feature case), CaptionSafetyStage returns a `pass_`. So results = [topic_pass, caption_pass_disabled], `[-1].passed` = True. Fine.

But consider: what if a future stage is *appended* that could pass when an earlier stage failed? No ? short-circuit prevents that. The `[-1]` is always either the first failure or final pass. So it's equivalent to aggregate `passed`.

However, there IS a fragility issue worth noting: `_evaluate_policy_stages` computes `_passed` (the correct aggregate) and **discards it**, forcing callers to reconstruct via `[-1].passed`. This is exactly redundant work that could diverge. But is it a *defect in this PR*? It's a

design/fragility concern. Let me verify the reasons.extend behavior ? this is a behavior change. In F2, a passing topic appended one reason. Now on a full pass, TWO reasons get appended (topic pass reason + caption pass reason). Let me check whether reasons semantics matter. Let me look at how reasons is used downstream.

15. user (2026-07-06T22:17:13.584Z)

```
276             text = _message_text(message)
277             result = engine_holder[ENGINE_KEY].evaluate(meta)
278             matched = result.matched
279             reasons = list(result.reasons)
280             if matched and source is not None:
281                 policy_results = await _evaluate_policy_stages(
282                     conn,
283                     user.id,
284                     source,
285                     _content_item_preview(
286                         user.id,
287                         source,
288                         message_id,
289                         document_id,
290                         meta,
291                         text,
292                         matched,
293                     ),
294                 )
295                 if policy_results:
296                     reasons.extend(result.reason for result in policy_results)
297                     matched = policy_results[-1].passed
298 
299             record = VideoRecord(
300                 user_id=user.id,
301                 channel_id=channel_id,
302                 message_id=message_id,
303                 text=text,
304                 document_id=document_id,
305                 duration_s=meta.duration_s,
306                 width=meta.width,
307                 height=meta.height,
308                 size_bytes=meta.size_bytes,
309                 mime_type=meta.mime_type,
310                 matched=matched,
311             )
312             try:
313                 inserted = await db.insert_video(conn, record)
314             except sqlite3.IntegrityError:
315                 # Lost a dedup race: a concurrent delivery of the same file
316                 # (same document_id or same message instance) won the insert.
317                 # The three-tier check above already classified this as new, so
318                 # the collision means another task is processing (or has
319                 # processed) it ? log and skip.
320                 log.info(
```

16. assistant (2026-07-06T22:17:18.470Z)

Now I understand the full flow. Let me note a subtle but important variable-shadowing issue in the `reasons.extend` generator: `reasons.extend(result.reason for result in policy_results)`. The loop variable is `result`, which **shadows** the outer `result` (the engine evaluation result from line 277). In a generator expression this is scoped, so it does NOT leak ? Python 3 generator expressions have their own scope. Let me verify this doesn't corrupt the outer `result`. Actually generator expressions create their own scope, so `result` inside is isolated. Let me confirm with a repro to be safe, and also test the config-parsing edges.

17. user (2026-07-06T22:17:21.765Z)

```
outer result after genexpr: OUTER
lst: ['a', 'b']
```

18. assistant (2026-07-06T22:17:24.764Z)

Good ? genexpr scope is isolated, no shadowing bug. Now let me run repros on the config-parsing edges: casefold substring false positives, empty-string term, missing vs empty caption, non-dict security_checks, and the round-trip consistency question.

19. user (2026-07-06T22:17:39.656Z)

```
Exit code 1
non-dict 'x': ValueError OK
non-dict 5: ValueError OK
non-dict ['a']: ValueError OK
non-dict True: ValueError OK
terms str: ValueError OK: security_checks.unsafe_caption_terms must be an array
empty/ws terms filtered -> ('ok',) enabled: True
only-empty terms enabled: False
dupes: ('a', 'a', 'a')
non-str item: ValueError OK
bad regex: ValueError OK
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\repro_parse.py", line 61, in <module>
    r = asyncio.run(run_stage(cfg_ass, "classic passage"))
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\repro_parse.py", line 55, in run_stage
    src = Source(kind="channel", id=1, title="t")
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 2 validation errors for Source
account_id
  Field required [type=missing, input_value={'kind': 'channel', 'id': 1, 'title': 't'},
input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
provider_source_id
  Field required [type=missing, input_value={'kind': 'channel', 'id': 1, 'title': 't'},
input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
```

20. assistant (2026-07-06T22:17:42.127Z)

Config parsing all validated. Let me fix the Source/ContentItem construction to test the stage behaviors.

21. user (2026-07-06T22:17:45.303Z)

```
SOURCE: ['id', 'account_id', 'provider', 'provider_source_id', 'title', 'kind', 'watched',
'seen_first_at', 'last_seen_at']
CI: ['id', 'account_id', 'source_id', 'provider', 'provider_message_id', 'content_type', 'text',
'link_url', 'media_ref', 'document_id', 'duration_s', 'width', 'height', 'size_bytes',
'mime_type', 'dedup_key', 'matched', 'seen_at']
```

```
---from tests---
37:        source=Source(
335:            Source(
30:        content_item=ContentItem(
351:        item = ContentItem(
```

22. user (2026-07-06T22:17:47.845Z)

```
25          policy_json: str = "{}",
26          source_title: str | None = None,
27          text: str | None = None,
28      ) -> PolicyContext:
29          return PolicyContext(
30              content_item=ContentItem(
31                  account_id=111,
32                  source_id=1,
33                  provider_message_id="42",
34                  text=text,
35                  dedup_key="tg:msg:1:42",
36              ),
37              source=Source(
38                  id=1,
39                  account_id=111,
40                  provider="telegram",
41                  provider_source_id="-1001",
42                  title=source_title,
43              ),
44              policy=Policy(
45                  id=1,
46                  account_id=111,
47                  name="default",
48                  config_json=policy_json,
49                  is_default=True,
50              ),
51          )
52
53
54      class RecordingStage:
55          def __init__(
56              self,
57              name: str,
58              calls: list[str],
59              *,
60              passed: bool = True,
61              reason: str | None = None,
62          ) -> None:
63              self.name = name
64              self.calls = calls
65              self.passed = passed
66              self.reason = reason or ("passed" if passed else "failed")
67
68          async def evaluate(self, ctx: PolicyContext) -> StageResult:
69              self.calls.append(self.name)
70              return StageResult(
71                  passed=self.passed,
72                  reason=self.reason,
73                  stage_name=self.name,
74                  metadata={"content_item_id": ctx.content_item.id},
75              )
76
77
78      async def test_evaluate_pipeline_passes_all_stages_in_order() -> None:
79          calls: list[str] = []
80          stages = [
81              RecordingStage("metadata", calls),
```

```
82              RecordingStage("topic", calls),
83          ]
84
```

23. user (2026-07-06T22:18:07.006Z)

```
substring 'ass' in 'classic passage': passed= False | unsafe caption term matched: ass
None caption: passed= True caption_present= False
ws caption: passed= True caption_present= False
ReDoS 25-char: passed=True elapsed=2.70s
topic-fail: agg passed= False results len= 1 [-1].passed= False stages= ['topic-preferences']
caption-fail: agg passed= False [-1].passed= False stages= ['topic-preferences', 'caption-
safety']
both-pass: agg passed= True [-1].passed= True reasons= ['no include topics configured', 'caption
passed safety checks']
```

24. assistant (2026-07-06T22:18:17.890Z)

All confirmed. Key results for my dimension:

1. **Aggregate equivalence**: `results[-1].passed` == aggregate `passed` in all three cases (topic-fail short-circuits to 1 result, caption-fail gives 2 results with `[-1]`=fail, both-pass gives `[-1]`=pass). The migration is semantically equivalent-or-better. Short-circuit preserves F2 topic behavior (caption doesn't run when topic fails ? confirmed `results len=1`).

2. **Fragility**: `_evaluate_policy_stages` computes the correct aggregate `_passed` and discards it, forcing callers to reconstruct `matched = policy_results[-1].passed`. This is correct only because `evaluate_pipeline` short-circuits. If short-circuiting were ever removed (stages run unconditionally), `[-1].passed` would reflect only the *last* stage, not the AND ? silently letting content through that an earlier stage failed. This is a latent design fragility.

3. **casefold substring false positive confirmed**: term `"ass"` matches `"classic passage"` ? fails. This is inherent to substring matching (documented behavior per PR spec), not a diff bug ? noting as low.

4. Config parsing edges all validate correctly (non-dict, non-list, non-string, empty-strip, bad-regex all raise ValueError ? fail-open; duplicates preserved; only-empty ? not enabled).

Let me verify the round-trip consistency question (does load_caption_safety_config read the same raw config as topic) and check the reasons behavior change is benign. Let me check how `matched` interacts ? one more concern: when topic stage FAILS, the caption reason is NOT added (only 1 reason). That's fine. But let me verify the `reasons` list semantic change doesn't break anything downstream, and confirm both loaders read identical raw config.

25. user (2026-07-06T22:18:20.968Z)

```
src/tg_video_filter/policy.py:186:            prefs = load_topic_preferences(ctx.policy)
src/tg_video_filter/policy.py:254:            config = load_caption_safety_config(ctx.policy)
src/tg_video_filter/policy.py:330:    "load_caption_safety_config",
src/tg_video_filter/policy.py:331:    "load_topic_preferences",
Binary file src/tg_video_filter/__pycache__/policy.cpython-313.pyc matches
---reasons downstream---
src/tg_video_filter/telethon_client.py:279:        reasons = list(result.reasons)
src/tg_video_filter/telethon_client.py:329:            "video_seen user=%s channel=%s msg=%s
matched=%s reasons=%s",
src/tg_video_filter/backfill.py:245:                reasons = list(result.reasons)
src/tg_video_filter/backfill.py:280:                    "backfill_video_seen user=%s channel=%s
msg=%s matched=%s reasons=%s",
```

26. assistant (2026-07-06T22:18:28.430Z)

Both loaders read from `ctx.policy.config_json` (same raw config), so consistency is preserved. `reasons` is only used for logging, so the behavior change (now two reasons on full pass instead

of one) is benign ? it's actually more informative logging.

Now, one subtle correctness concern I want to verify: the `matched = policy_results[-1].passed` when the caption stage passes but returns `caption_present=False` (no caption) ? does the topic stage result get overwritten incorrectly? No: if topic passes and caption passes (disabled or no caption), `[-1]`=caption pass ? matched=True. Correct.

But there's a more important asymmetry to check: **the `matched` field on the persisted record vs the topic-only F2 behavior**. In F2, `matched = topic_result.passed`. Now, even when caption safety is *disabled* (the dormant default), `matched = policy_results[-1].passed` = caption's pass. Since caption returns pass when disabled, `matched` still equals topic's passed value (topic must have passed for caption to run). Confirmed equivalent. Let me run the full test suite for this file to ensure nothing regressed, then finalize.

27. user (2026-07-06T22:18:33.304Z)

```
src\tg_video_filter\delivery.py                 102     80     48      0    15%   40-48, 53-58,
63-65, 87-95, 107-121, 145-160, 177-192, 218-278
src\tg_video_filter\filters\__init__.py          39     26     18      0    23%   32-33, 42-52,
62-75
src\tg_video_filter\filters\base.py              26      2      0      0    92%   48, 52
src\tg_video_filter\filters\content_type.py      23     15     12      0    23%   18-22, 29-33,
40-44
src\tg_video_filter\filters\duration.py          15     10      6      0    24%   17-18, 21-28
src\tg_video_filter\filters\resolution.py        13      8      6      0    26%   16, 21-27
src\tg_video_filter\filters\size.py              18     11      6      0    29%   18, 21-29, 34
src\tg_video_filter\manager.py                   36     36      8      0     0%   26-204
src\tg_video_filter\models.py                   163     10      8      1    90%   363-367, 377,
417, 425-427
src\tg_video_filter\policy.py                   156      8     56      8    92%   65, 99, 124,
152, 160, 172, 174->170, 211, 221
src\tg_video_filter\telethon_client.py           80     46     32      2    32%   79-117, 359,
372-377, 382-388, 401-402, 438, 454-459, 471->exit
-------------------------------------------------------------------------------------
TOTAL                                          1393    714    380     21    42%
FAIL Required test coverage of 80.0% not reached. Total coverage: 41.85%
22 passed in 1.27s
```

28. assistant (2026-07-06T22:18:37.138Z)

22 passed (the coverage fail is just because I ran a single file, not the full suite ? the reviewer confirmed 523 passed @ 95.77% on the full gate). My dimension analysis is complete.

Let me summarize findings. The migration is correct-but-fragile. The main defect worth reporting is the fragility of `matched = policy_results[-1].passed` combined with `_evaluate_policy_stages` discarding the already-computed correct `passed` aggregate.

29. user (2026-07-06T22:18:56.506Z)

Structured output provided successfully